Java Mini Project Coding Guidelines

Beginner Manual Review Guide

Java Desktop Application using Swing UI and JDBC

These guidelines define the coding practices expected during Code Review 1 and Code Review 2. They are intentionally practical and suitable for students who are still learning Java.

1 Purpose and review approach

The purpose of code review is to help the team produce code that is correct, readable, modular and easy to improve. Reviewers should identify specific issues, explain why they matter, and suggest an achievable correction. The review is a learning activity, not a fault-finding exercise.

Manual review rule: Automated code-analysis, code-quality scoring and code-review tools shall not be used. Reviewers shall read the source code, compile and run the project, manually test the user interface and database operations, and discuss the code with its authors.

1.1 What reviewers may do

- Read the source files and follow the program flow manually.
- Compile and run the application and use its major Swing screens.
- Enter valid and invalid data and observe messages and database results.
- Ask the author to explain a class, method, SQL operation or design choice.
- Record findings using the peer-review form and verify the correction later.

1.2 Project baseline

1.3 Required review attitude

- Comment on the code, not the person who wrote it.
- Mention the class and method so that the finding can be located easily.
- Separate errors from style suggestions and optional improvements.
- Acknowledge code that is clear or well structured, not only defects.

2 Naming formatting and readability

2.1 Naming conventions

2.2 Formatting rules

- Use one consistent indentation style throughout the project; four spaces is recommended.
- Place opening braces consistently and align closing braces with the statement that opened the block.
- Write one statement per line and use blank lines to separate logical sections.
- Keep lines reasonably short; wrap long method calls or conditions in a readable way.
- Remove commented-out code, unused variables, duplicate imports and temporary debug messages before review.
- Use spaces around operators and after commas so expressions are easy to read.

2.3 Comments

- Use comments to explain purpose, assumptions or a non-obvious decision.
- Do not write comments that merely repeat an obvious line of code.
- Update or remove comments when the related code changes.
- Use TODO only for a specific unfinished task and include the responsible person or intended action.

2.4 Readability check

A beginner reviewer should be able to identify what a class represents, what a method does, what information a variable stores, and what should happen when an operation fails.

3 Methods classes and code organisation

3.1 Methods

- Give each method one clear task. Split a method when it validates input, controls the UI, performs SQL and formats output together.
- Use parameters and return values rather than depending on unrelated global or static state.
- Avoid repeating the same block of code in multiple places; extract a common method when appropriate.
- Keep the number of parameters manageable. If many values always travel together, consider a model object.
- Use early validation or clear conditions so deeply nested if statements are avoided.
- Do not ignore a returned value when it indicates success, failure or a required result.

3.2 Classes

- Each class should have one understandable responsibility.
- Keep fields private unless there is a clear reason to expose them.
- Use constructors to create valid objects and initialise required fields.
- Keep Swing components in UI classes, business rules in service or model classes, and SQL in DAO classes.
- Avoid one very large class that controls every screen, calculation and database operation.
- Avoid utility classes filled with unrelated static methods.

3.3 Project organisation

3.4 Values and constants

- Replace repeated fixed values with named constants when the value has a clear meaning.
- Do not hard-code database credentials, file paths or environment-specific settings inside multiple classes.
- Avoid unexplained magic numbers in conditions and calculations.

4 Object-oriented programming guidelines

4.1 Encapsulation

- Keep object fields private and provide only the operations needed to use or update the object safely.
- Validate important values before changing object state.
- Do not provide setters for every field automatically if a field should not change freely.

4.2 Abstraction and interfaces

- Expose simple operations that describe what a component does without forcing other classes to know all implementation details.
- Use an interface when more than one implementation is meaningful or when it clearly separates a dependency.
- Do not create interfaces only to increase the number of Java concepts used.

4.3 Inheritance composition and polymorphism

4.4 Access modifiers

- Use private for implementation details and fields by default.
- Use public only for operations that other packages or classes genuinely need.
- Use protected and package access deliberately; do not choose them only to make an error disappear.

4.5 OOP warning signs

- Public fields are modified directly from many classes.
- Most program data is stored in unrelated static variables.
- One class performs UI, validation, calculation and SQL operations.
- Inheritance is used only to reuse a few lines of code without a true relationship.

5 Input validation and exception handling

5.1 Input validation

- Validate required fields before starting an operation.
- Check number, date and range rules before converting or saving values.
- Check business rules in a service or model class as well as basic format rules in the UI.
- Prevent duplicate or invalid database records using both program checks and database constraints where suitable.
- Show a clear message that tells the user what must be corrected.

5.2 Exception handling

- Catch an exception only where the program can handle it, convert it into a meaningful result, or add useful context.
- Do not use an empty catch block or silently ignore a failure.
- Do not show a full technical stack trace to an ordinary application user.
- Use finally or try-with-resources when a resource must be closed.
- Keep the original cause when one exception is converted into another.
- Avoid catching Exception everywhere when a more specific exception can be handled.

5.3 Manual failure checks

5.4 Logging and messages

- User messages should be short, polite and actionable.
- Technical details may be written to a log or console during development, but remove uncontrolled debug printing before review.

6 Swing user interface guidelines

- Use meaningful titles, labels and button text. A user should understand each screen without reading the source code.
- Prefer layout managers to a null layout and repeated setBounds calls so the interface remains usable across sizes.
- Keep related fields together and maintain a consistent order, spacing and alignment.
- Validate input before calling the service or DAO layer.
- Keep event-listener code short; delegate business work to another class.
- Do not place SQL statements inside JFrame, JPanel or dialog classes.
- Use confirmation messages for destructive or important actions.
- Show success, empty-result and error states clearly.
- Use JTable or another suitable component for collections of records rather than many unrelated labels.
- Start the Swing interface on the Event Dispatch Thread using SwingUtilities.invokeLater where applicable.

6.1 Screen review table

6.2 Responsiveness

Operations that may take noticeable time should not make the interface appear permanently frozen. For this mini project, reviewers should at least identify slow database or file operations and ask the team to provide visible progress or a safe waiting state when necessary.

7 JDBC and database coding guidelines

- Keep SQL and record-mapping code inside focused DAO classes.
- Use PreparedStatement for values supplied by users or variables; do not build SQL by joining user input into a string.
- Use try-with-resources to close Connection, PreparedStatement and ResultSet objects safely.
- Use clear DAO method names such as findById, findAll, insert, update and deleteById.
- Return useful results to the service layer; do not update Swing components directly from a DAO.
- Use transactions for related changes that must either all succeed or all fail.
- Define primary keys, foreign keys, unique constraints and nullability in the database where appropriate.
- Avoid select * when only a few columns are needed and keep column mapping easy to verify.
- Keep connection settings in one configuration location and do not repeat credentials throughout the code.

7.1 Manual JDBC review

7.2 Database verification

- Run create, read, update and delete operations that are part of the approved requirements.
- Check the database after the UI reports success.
- Try invalid and duplicate values and confirm that the database remains consistent.
- Reopen the application and verify that saved information can be retrieved.

8 Code Review 1 expectations

Code Review 1 carries 5 marks and focuses on basic Java, object-oriented programming and coding standards. A partially implemented project may be reviewed, but the submitted functions must compile, run and demonstrate a clear project structure.

8.1 Minimum evidence

- Current source code and clear instructions to run the project.
- Working basic navigation and at least one meaningful end-to-end operation.
- Classes separated into appropriate responsibilities or packages.
- Use of variables, conditions, loops, methods, constructors and access modifiers where appropriate.
- Evidence of encapsulation and other relevant OOP concepts.
- Internal peer-review record and review by the faculty-assigned team.
- Updated HackerRank profile and Java stars for every member.

8.2 Manual checklist

[ ] The submitted project compiles and starts.

[ ] Implemented functions give the expected result for normal input.

[ ] Names and formatting are consistent and readable.

[ ] Methods have clear tasks and avoid obvious duplication.

[ ] Classes demonstrate suitable encapsulation and responsibility separation.

[ ] Swing classes do not contain database SQL.

[ ] DAO or database code uses JDBC safely for the implemented operation.

[ ] Invalid input does not cause an uncontrolled crash.

[ ] Each team member can explain the code they contributed.

8.3 Not required yet

Complete JavaDoc coverage and sophisticated advanced Java features are not required for Code Review 1. However, students should begin documenting important decisions and should not postpone obvious refactoring or error handling.

9 Code Review 2 expectations

Code Review 2 carries 7 marks and examines how the team improved the first review submission. The focus is on appropriate advanced Java concepts, reliable handling of resources and errors, proper JavaDocs and consistent coding style.

9.1 Appropriate advanced concepts

Advanced features should solve a real problem in the project. Students are not expected to force every item below into the code.

9.2 Improvement expected after Code Review 1

- Important findings from the first review are fixed or addressed with a clear reason.
- Duplicated, confusing or overlong code is refactored.
- Exception handling, validation and JDBC resource management are complete for implemented features.
- Naming, formatting and package organisation are consistent across the team.
- The application demonstrates more than isolated screens; important workflows are integrated.

9.3 Manual checklist

[ ] Advanced concepts are relevant and can be explained by the team.

[ ] Collections and generic types are used safely where needed.

[ ] JDBC resources and transactions are handled correctly.

[ ] Exceptions are not ignored and user messages are meaningful.

[ ] JavaDocs cover the required classes and methods and match the code.

[ ] Review findings are recorded, corrected and rechecked.

10 JavaDoc and documentation guidelines

JavaDocs are mandatory in Code Review 2 for important classes, interfaces, constructors and public or protected methods. Private methods should be documented when their purpose or assumptions are not obvious.

10.1 What a useful JavaDoc explains

- The responsibility of a class or interface.
- What a method does from the caller's point of view.
- The meaning and valid range of each parameter when it is not obvious.
- What the method returns, including important special values.
- Exceptions that the caller may need to handle.
- Important side effects, preconditions or database changes.

10.2 Required tags

10.3 JavaDoc quality check

- The description matches the current method signature and behaviour.
- Parameter, return and exception information is complete where required.
- The wording explains the contract rather than repeating the method name.
- Generated documentation would not contain empty or misleading descriptions.
- Authors update the JavaDoc whenever the method contract changes.

10.4 Other project documentation

- Keep short run instructions, database setup information and required configuration with the project.
- Record major known limitations honestly.
- Use the same names for modules, classes, screens and requirements in documents and code.

11 Manual peer-review procedure

11.1 Internal review by the same team

- Each member reviews code mainly written by another member.
- The reviewer records the class or method, finding, importance and suggested correction.
- The author responds, makes the correction where accepted, and records the status.
- Another team member rechecks important corrections before submission.

11.2 External review by the assigned team

- The assigned team performs an independent review using the same rubric.
- External reviewers should not rewrite the code for the author team.
- Findings must refer to observable code or behaviour and should include an achievable suggestion.
- The author team records whether each finding was fixed, accepted for later work or declined with a reason.

11.3 Finding categories

11.4 Final submission check

[ ] The project version reviewed is identified.

[ ] Internal and external reviewers are named.

[ ] All Must Fix findings have a recorded response and recheck result.

[ ] Accepted changes are present in the submitted code.

[ ] Every member's HackerRank profile and current Java stars are included.